

THERE ARE NO BIG
PROBLEMS, THERE
ARE JUST A LOT OF
LITTLE PROBLEMS.

Henry Ford

PICTUREQUOTES.COM

Good
Evening ☺

Today's content

01. Time complexity of Recursive code
02. Space complexity of Recursive code
03. K^n symbol

Time complexity in Recursion

01. Identify the Recurrence relation/Recursive relation
02. Generalised the relation for k steps
03. Try to reduce the right side function.

Q = Sum of n natural numbers

```
int sum (int n){  
    if (n==1) return 1;  
    return (sum(n-1) + n);  
}
```

Assumption
// Time taken to calculate
 $\text{sum}(n) \rightarrow f(n)$
 $f(n-1)$

$$f(n) = f(n-1) + O(1)$$

$$f(n) = f(n-1) + 1$$

$$f(1) = 1$$

01. $f(n) = f(n-1) + 1$ ← After 1st substitution

$$\begin{aligned} f(n-1) &= f(n-1-1) + 1 \\ &= f(n-2) + 1 \end{aligned}$$

$$f(n) = f(n-2) + 1 + 1$$

$$f(n) = f(n-2) + 2 \quad \leftarrow \text{After } 2^{\text{nd}} \text{ sub}$$

$$f(n-2) = f(n-2-1) + 1$$

$$= f(n-3) + 1$$

$$f(n) = f(n-3) + 1 + 2$$

$$f(n) = f(n-3) + 3 \quad \leftarrow \text{After } 3^{\text{rd}} \text{ sub}$$

⋮

After kth step

$$f(n) = f(n-k) + k$$

$$f(1) = 1$$

$$n-k=1$$

$$k = n-1$$

$$f(n) = f(1) + n-1$$

$$= x + n-x = n$$

$$TC = O(n)$$

02. `int fact(n) {` $n > 0$
 | `if (n == 1) { return 1; }`
 | `return fact(n-1) * n`
 | $f(n-1)$
 3

Time taken to calculate $\text{fact}(n) = f(n)$

$$f(n) = f(n-1) + O(1)$$

$$f(n) = f(n-1) + 1$$

$$f(1) = 1$$

01. $f(n) = \underline{f(n-1)} + 1$

$$f(n-1) = f(n-1-1) + 1$$

$$f(n-1) = f(n-2) + 1$$

$$f(n) = f(n-2) + 1 + 1$$

$$f(n) = f(n-2) + 2$$

$$f(n-2) = f(n-2-1) + 1$$

$$f(n-2) = f(n-3) + 1$$

$$f(n) = f(n-3) + 1 + 2$$

$$f(n) = f(n-3) + 3$$

⋮

After k^{th} substitution

$$f(n) = f(n-k) + k$$

$$f(1) = 1$$

$$n-k=1$$

$$k = n-1$$

$$f(n) = f(1) + n-1$$

$$= 1 + n-1$$

$$f(n) = O(n)$$

$$TC = O(n)$$

03. int fact(n) { $n \geq 0$

if (n == 0) { return 1; }
return fact(n-1) * n
3 $f(n-1)$

Time taken to calculate fact(n) = $f(n)$

$$f(n) = f(n-1) + O(1)$$

$$f(n) = f(n-1) + 1$$

$$f(0) = 1$$

01. $f(n) = \underline{f(n-1)} + 1$

$f(n-1) = f(n-1-1) + 1$

$f(n-1) = f(n-2) + 1$

$$f(n) = f(n-2) + 1 + 1$$

$$f(n) = f(n-2) + 2$$

$f(n-2) = f(n-2-1) + 1$

$f(n-2) = f(n-3) + 1$

$$f(n) = f(n-3) + 1 + 2$$

$$f(n) = f(n-3) + 3$$

⋮

After k^{th} substitution

$$f(n) = f(n-k) + k$$

$$f(0) = 1$$

$$n - k = 0$$

$$\underline{\underline{n = k}}$$

$$f(n) = f(0) + n$$

$$= 1 + n$$

$$T_C = O(n)$$

03.

```
int pow1(a, n)
```

```
    if (n == 0) return 1
```

```
    return pow1(a, n-1) * a;
```

3

TODO

$T_C: O(n)$

```

04. int pow3(a, n) {
    if (n == 0) return 1
    p = pow3(a, n/2)
    if (n % 2 == 0) {
        return p * p
    } else {
        return p * p * a
    }
}

```

// Time taken to calculate

$$\text{pow3}(a, n) = f(n)$$

$$f(n) = f(n/2) + 1$$

$$f(0) = 1$$

$$f(1) = 1$$

$$01. f(n) = f(n/2) + 1 \rightarrow f\left(\frac{n}{2^1}\right) + 1$$

$$f(n/2) = f(n/4) + 1$$

$$02. f(n) = f(n/4) + 2 \rightarrow f\left(\frac{n}{2^2}\right) + 2$$

$$f(n/4) = f(n/8) + 1$$

$$03. f(n) = f(n/8) + 3 \rightarrow f\left(\frac{n}{2^3}\right) + 3$$

⋮

After k^{th} substitution

$$f(n) = f\left(\frac{n}{2^k}\right) + k$$

$$f(0) = 1$$

$$f(1) = 1$$

$\frac{n}{2^k} = 0 \rightarrow$ NO answer for this

$$\frac{n}{2^k} = 1$$

$$n = 2^k$$

$$k = \log_2 n$$

$$f(n) = f(1) + \log_2 n$$

$$= 1 + \log_2 n$$

$$TC = O(\log_2 n)$$

as.

```
long pow2 (int a, int n){
    if (n==1) return a
    if (n==0) return 1L;
    if (n/2 == 0)
        return pow2(a, n/2) * pow2(a, n/2);
    else
        return pow2(a, n/2) * pow2(a, n/2) * a;
}
```

// Assumption = $\text{pow2}(a, n)$ is going to take $f(n)$ time

$$f(n) = f(n/2) + f(n/2) + O(1)$$

$$f(n) = 2f(n/2) + 1$$

$$f(0) = 1$$

$$f(1) = 1$$

$$01. \quad f(n) = 2f(n/2) + 1 \quad = \quad 2^1 f\left(\frac{n}{2^1}\right) + 2^1 - 1$$

$$f(n/2) = 2f(n/4) + 1$$

$$02 \quad f(n) = 2 * (2f(n/4) + 1) + 1$$

$$= 4f(n/4) + 2 + 1$$

$$f(n) = 4f(n/4) + 3 = 2^2 f\left(\frac{n}{2^2}\right) + 2^2 - 1$$

$$f(n/4) = 2f(n/8) + 1$$

$$f(n) = 4 * (2f(n/8) + 1) + 3$$

$$= 8f(n/8) + 4 + 3$$

$$f(n) = 8f\left(\frac{n}{8}\right) + 7 = 2^3 f\left(\frac{n}{2^3}\right) + 2^3 - 1$$

$$f\left(\frac{n}{8}\right) = 2f\left(\frac{n}{16}\right) + 1$$

$$f(n) = 8 * \left(2f\left(\frac{n}{16}\right) + 1\right) + 7$$

$$= 16f\left(\frac{n}{16}\right) + 8 + 7$$

$$f(n) = 16f\left(\frac{n}{16}\right) + 15 = 2^4 f\left(\frac{n}{2^4}\right) + 2^4 - 1$$

⋮

After k^{th} substitution

$$f(n) = 2^k f\left(\frac{n}{2^k}\right) + 2^k - 1$$

$$f(0) = 1$$

$$f(1) = 1$$

$$\frac{n}{2^k} = 1 \Rightarrow n = 2^k$$

$$k = \log_2 n$$

$$f(n) = n * f(1) + n - 1$$

$$= n * 1 + n - 1$$

$$= n + n - 1$$

$$TC = O(n)$$

$$f(n) = 2n - 1$$

* Space complexity

Obs 1 → Function calls are stored in stack
& hence it is going to consider as an
extra space

SC: Size of the stack

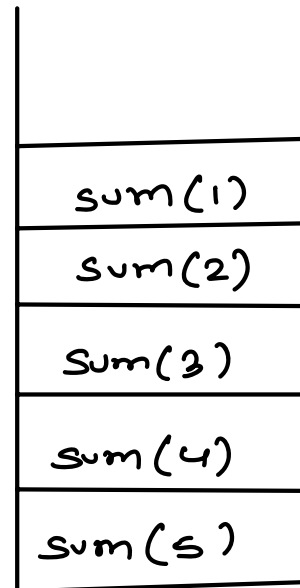
```
int sum (int n){  
    if (n==1) return 1  
    return (sum(n-1)+n)  
}
```

$n=5$

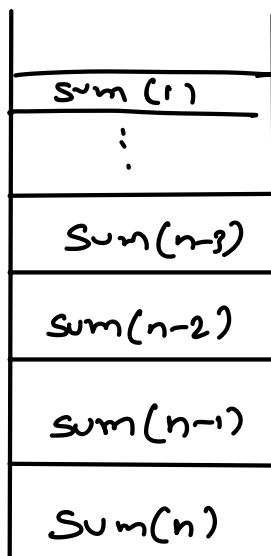


5 extra spaces

main()



n



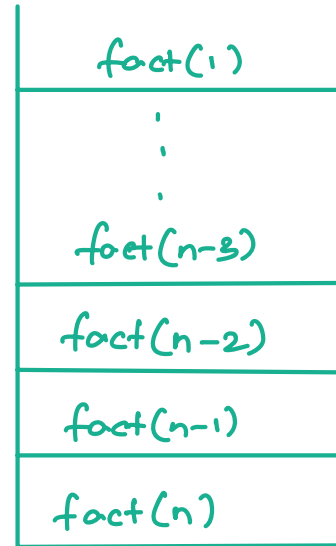
$O(n)$ space

02. `int fact(n){`

`if (n==1){return 1}`

`return fact(n-1)*n`

`}`



$O(n)$
space

04. `int pow3(a,n){`

`if (n==0) return 1`

`p = pow3(a, n/2)`

`if (n%2 == 0){`

`return p*p`

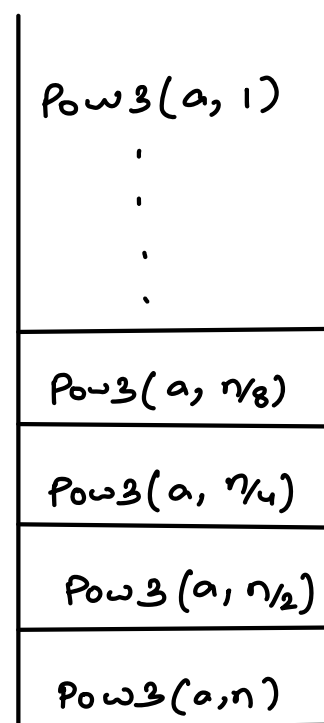
`}`

`else {`

`return p*p*a`

`}`

`}`



$\log n$
calls

SC: $O(\log n)$

05

int fib(n)

if (n ≤ 1) return n

return fib(n-1) + fib(n-2)

3

// Assumption → Time taken to calculate fib(n) → f(n)

$$f(n) = f(n-1) + f(n-2) + 1$$

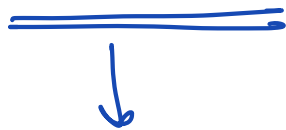
$$f(n-1) = f(n-1-1) + f(n-1-2) + 1$$

$$f(n-1) = f(n-2) + f(n-3) + 1$$

$$f(n-2) = f(n-3) + f(n-4) + 1$$

$$f(n) = f(n-2) + f(n-3) + 1 + f(n-3) + f(n-4) + 1 + 1$$

Not a good approach as it is lengthy



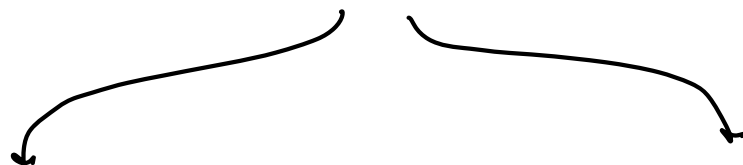
Bad approach → Multiple calls

Total calls

level = 0

fib(n)

2^0



2^1

level = 1

fib(n-1)

fib(n-2)



2^2

lev = 2

fib(n-2)

fib(n-3)

fib(n-3)

fib(n-4)

2^3

lev 3

⋮

2^n

lev n

$$\text{Total fn calls} = 2^0 + 2^1 + 2^2 + 2^3 + \dots + 2^n$$

$$\text{Sum of GP} = \frac{a(r^n - 1)}{r - 1}$$

$$a = 1$$

$$r = 2$$

$$n = n + 1$$

$$\text{Total os} = \frac{1 * (2^{n+1} - 1)}{2 - 1}$$

$$= 2^{n+1} - 1$$

$$\approx 2^n$$

$$\text{TC: } O(2^n)$$

OS

```
int fib(n)
```

```
    if (n ≤ 1) return n
```

```
    return fib(n-1) + fib(n-2)
```

3

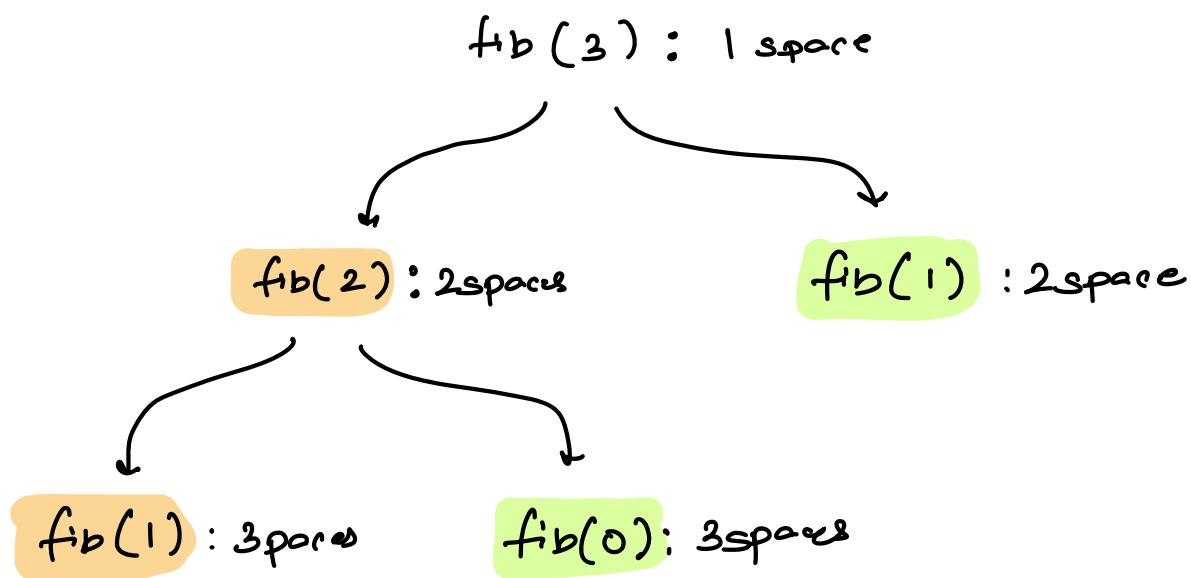
Space complexity

↳ if you're making a tree then

SC: height of the tree

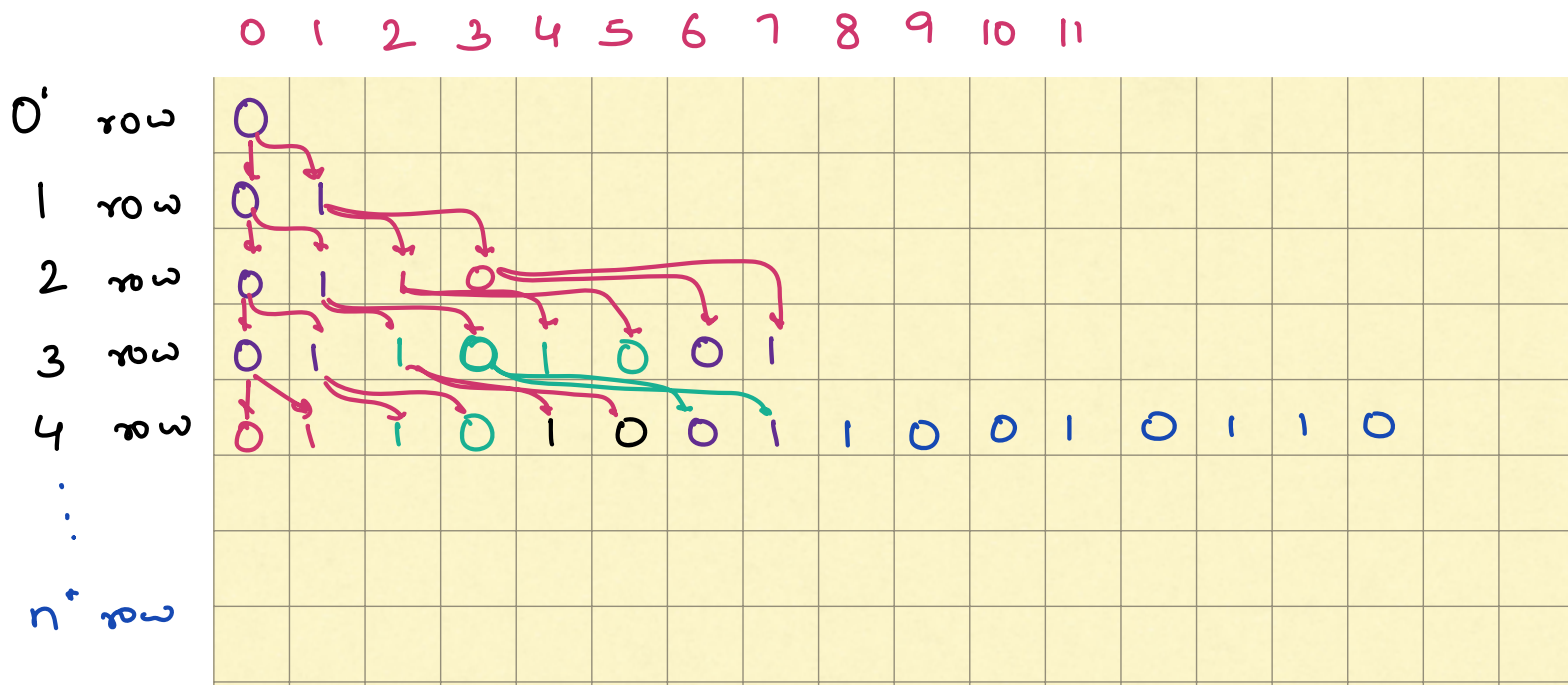
SC: $O(n)$

$n=3$



K⁺ symbol

Given N, k . Find the k^{th} ele in N^{th} row.

$$0 \rightarrow \{0, 1\} \quad 1 \rightarrow \{1, 0\}$$


1<

3

4

→ Ans = 1

2

2

→ $A_{22} = 1$

3

6

$\rightarrow A_1 = 0$

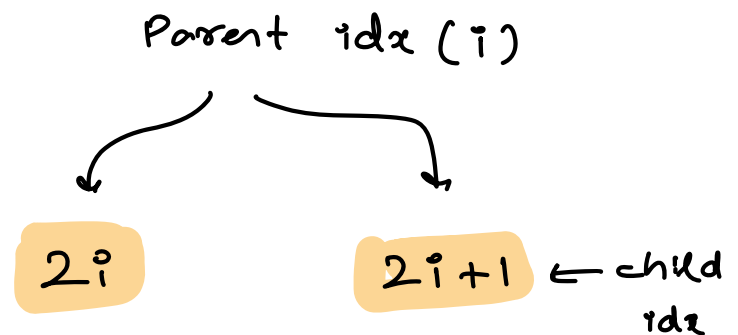
Brute force = N $\rightarrow$ $(N-1)$ $\rightarrow$ $(N-2)$ $\rightarrow$ $(N-3) \dots 0$

Total calls = $2^n + (2^{n-1}) + (2^{n-2}) + (2^{n-3}) + \dots + 2^0$

$$TC = O(2^n)$$

Observation $\rightarrow$ Every no. is going to generate two child no.

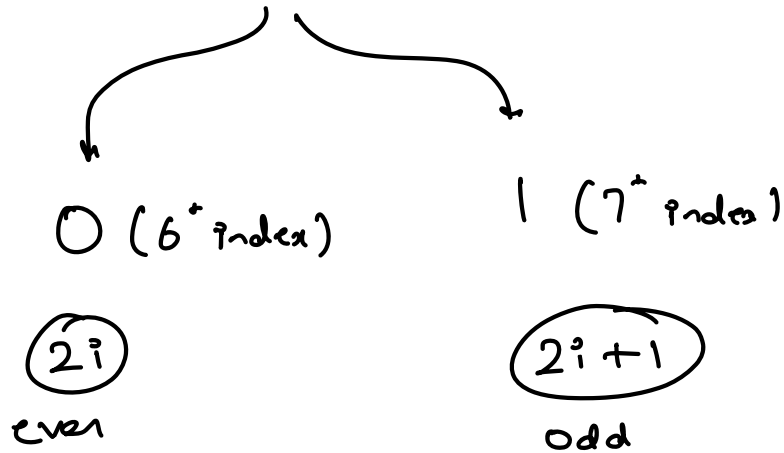
<u>idx</u>	<u>child idx</u>
0	0, 1
1	2, 3
3	6, 7
2	4, 5



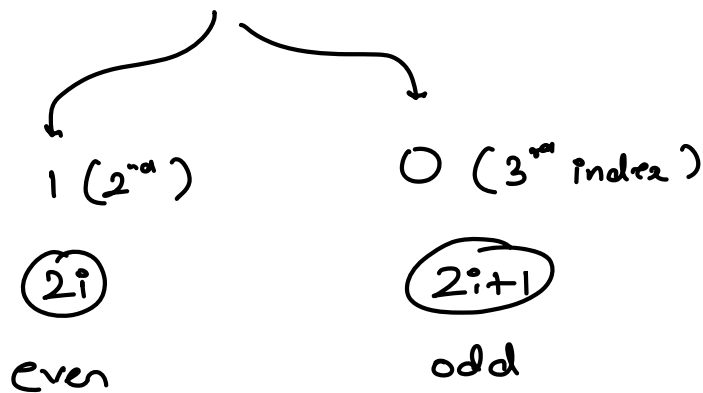
child idx $\left(\frac{i}{2} \right) \rightarrow \frac{i}{2}$

Observation 2

Parent value = 0 (3rd index)



Parent value = 1 (1st index)



⇒ $2i^{\text{th}}$ indexed child will always be having same value as parent

⇒ $(2i+1)^{\text{th}}$ indexed child will have different value than the parent

→ Ans = 1

$n=3$

$k=4$

fun (int n , int k) $n=3$, $k=4$

if ($n==0$ || $k==0$) return 0

int p = fun ($n-1$, $k/2$)

if ($k \% 2 == 0$) return p

else return $1-p$;

}

1

fun (int n , int k) $n=2$, $k=2$

if ($n==0$ || $k==0$) return 0

int p = fun ($n-1$, $k/2$)

if ($k \% 2 == 0$) return p

else return $1-p$;

}

return 1

fun (int n , int k) $n=1$, $k=1$

if ($n==0$ || $k==0$) return 0

int p = fun ($n-1$, $k/2$)

if ($k \% 2 == 0$) return p

else return $1-p$;

}

0

```

fun (int n, int k) n=0, k=0
|
| if (n==0 || k==0) return 0
|   int p = fun(n-1, k/2)
|     if (k%2 == 0) return p
|     else return 1-p;
|
?

```

Doubts

```
int function()
```

```
| if (n%2 == 0) return 0 ✗
```

```
| → return function(n-1) + function(Math.floor(n/2));
```

3

odd

n=71

function(71-1)

← + function(35)

0

even

n=6

→ O(1)

function(70)

